

JavaScript: масиви та об'єкти

Головна ідея: масив (`[]`) зберігає впорядковану колекцію елементів за індексами; об'єкт (`{}`) зберігає дані як пари ключ-значення. Обидва — складні типи: присвоюються **за посиланням**, а не за значенням, як примітиви. Це впливає на копіювання, порівняння та зміну даних.

1. Масиви: основи

```
const planets = ['Earth', 'Mars', 'Venus']; // літерал масиву — квадратні дужки

planets[0];           // 'Earth' — доступ за індексом (з 0)
planets[planets.length - 1]; // 'Venus' — останній елемент
planets.length;       // 3 — кількість елементів (динамічна властивість)

planets[0] = 'Jupiter'; // перевизначення елемента за індексом
planets[3];             // undefined — неіснуючий індекс, без помилки
```

2. Присвоєння за посиланням vs за значенням

	Примітиви (Number, String, Boolean...)	Складні типи (Array, Object)
Копіювання	за значенням — повна незалежна копія	за посиланням — копіюється лише "адреса" в пам'яті
Зміна копії	оригінал не змінюється	оригінал змінюється теж (обидві змінні вказують на одні дані)

```
let a = 5;
let b = a;
a = 10;
console.log(b); // 5 — незалежна копія

const arr1 = ["Mango", "Poly"];
const arr2 = arr1; // посилання на той самий масив
arr1[1] = "Jacob";
console.log(arr2); // ["Mango", "Jacob"] — змінився теж!

const x = [1, 2, 3];
const y = [1, 2, 3];
console.log(x === y); // false — різні масиви в пам'яті, навіть з однаковим вмістом
```

3. Приведення типів масиву

Приведення	Приклад	Результат
Масив → рядок	<code>String([1, true, "Poly"])</code>	<code>"1,true,Poly"</code> (елементи через кому)
Масив → число	<code>Number([1,2,3])</code> , <code>Number([])</code>	<code>NaN</code> , <code>0</code>
Масив → буль	<code>Boolean([])</code> , <code>Boolean([1,2])</code>	<code>true</code> (завжди, навіть порожній!)

4. Методи масиву

Метод	Що робить	Приклад
<code>.join(delim)</code>	об'єднує елементи в рядок через роздільник	<code>["a","b"].join("-")</code> → <code>"a-b"</code>
<code>.slice(start, end)</code>	копія частини масиву (end не включно), оригінал не змінює	<code>[1,2,3,4].slice(1,3)</code> → <code>[2,3]</code>
<code>.concat(arr2, ...)</code>	об'єднує масиви в новий, оригінали не змінює	<code>[1,2].concat([3,4])</code> → <code>[1,2,3,4]</code>
<code>.indexOf(elem)</code>	індекс першого входження або <code>-1</code> (сувора рівність)	<code>["a","b"].indexOf("b")</code> → <code>1</code>
<code>.push(el1, el2...)</code>	додає елемент(и) в кінець, змінює оригінал	<code>arr.push("x")</code>
<code>.includes(elem)</code>	чи є елемент у масиві — <code>true</code> / <code>false</code>	<code>[1,2,3].includes(2)</code> → <code>true</code>

slice() vs split(): `slice()` — метод масиву (і рядка) для копіювання частини. `split(delim)` — метод **рядка**, розбиває рядок на масив за роздільником: `"a-b-c".split("-")` → `["a","b","c"]`. Це протилежна операція до `join()`.

5. Ітерація по масиву

```
// Класичний for — є доступ до індексу i
for (let i = 0; i < planets.length; i += 1) {
  console.log(planets[i]);
}

// for...of — простіше, коли індекс не потрібен
for (const planet of planets) {
  console.log(planet);
}
```

`for...of` завжди перебирає весь масив від початку до кінця; дострокове завершення — тільки через `break`. Немає доступу до лічильника ітерації.

6. Об'єкти: основи

```
const book = {
  title: "The Last Kingdom", // ключ: значення
  author: "Bernard Cornwell",
  genres: ["adventure"],      // значенням може бути масив
  rating: 8.38,
};

book.title; // доступ через крапку — коли ключ відомий заздалегідь
book["title"]; // доступ через дужки — коли ключ у змінній/невідомий наперед
book.newProp = "x"; // додавання нової властивості (так само, як і зміна існуючої)
```

Короткі та обчислювані властивості

```
// Коротка властивість — якщо ім'я змінної == імені властивості
const name = "Henry";
const user = { name }; // замість { name: name }

// Обчислювана властивість — ключ береться зі значення змінної
const propName = "email";
const creds = { [propName]: "a@mail.com" }; // { email: "a@mail.com" }
```

7. Перебір об'єкта

Об'єкт — не ітерабельна сутність: `for`, `for...of` і масивні методи на ньому не працюють.

Спосіб	Синтаксис	Повертає
<code>for...in</code>	<code>for (const key in obj) {...}</code>	перебирає ключі; значення — <code>obj[key]</code>
<code>Object.keys(obj)</code>	—	масив ключів (імен властивостей)
<code>Object.values(obj)</code>	—	масив значень властивостей

```
const keys = Object.keys(book); // ["title", "author", "genres", "rating"]
const values = Object.values(book); // ["The Last Kingdom", "Bernard Cornwell", ["adventure"], 8.38]

for (const key of keys) {
  console.log(key, book[key]);
}
```

8. Масив об'єктів

Типова структура: колекція однотипних сутностей (усі об'єкти мають однаковий набір ключів, різні значення).

```
const products = [
  { name: "Radar", price: 1300, quantity: 4 },
  { name: "Droid", price: 400, quantity: 7 },
];

// Пошук об'єкта за значенням властивості
function getPrice(productName) {
  for (const product of products) {
    if (product.name === productName) return product.price;
  }
  return null;
}

// Збір значень однієї властивості з усіх об'єктів
const names = [];
for (const product of products) {
  names.push(product.name);
} // ["Radar", "Droid"]
```

9. Методи об'єкта та this

```
const bookShelf = {
  books: ["The Last Kingdom"],
  getBooks() { // метод — функція як властивість об'єкта
    return this.books; // this — посилання на об'єкт, що викликав метод
  },
  addBook(bookName) {
    this.books.push(bookName); // зміна масиву за посиланням, this.books
  },
};

bookShelf.addBook("The Mist");
bookShelf.getBooks(); // ["The Last Kingdom", "The Mist"]
```

this завжди вказує на об'єкт зліва від крапки під час виклику методу — не на ім'я об'єкта, записане в коді. Це важливо, бо той самий метод може викликатись у різних об'єктів.

Зміна об'єкта всередині масиву

```
changeRating(bookName, newRating) {
  for (const book of this.books) {
    if (book.title === bookName) {
      book.rating = newRating; // book – посилання на потрібний об'єкт у масиві
    }
  }
}
```

10. Spread і Rest (синтаксис `...`)

Однаковий символ `...`, протилежні дії: **rest** — збирає аргументи в масив; **spread** — розкладає масив/об'єкт на окремі елементи.

Rest — залишкові параметри (у функції)

```
function add(...args) { // всі аргументи збираються в масив args
  let sum = 0;
  for (const item of args) sum += item;
  return sum;
}
add(1, 2, 3); // 6

function multiply(first, second, ...rest) { // rest – завжди останній параметр
  console.log(first, second, rest); // 1 2 [3, 4]
}
multiply(1, 2, 3, 4);
```

Spread — розкладання (при виклику функції або створенні масиву/об'єкта)

```
const temps = [14, -4, 25];
Math.max(...temps); // 25 – масив розкладається в окремі аргументи

const copy = [...temps]; // незалежна копія масиву
const merged = [...arr1, ...arr2]; // об'єднання масивів (як concat)

const settings = { ...defaults, ...overrides }; // об'єднання об'єктів;
// властивості з overrides перезаписують однойменні з defaults
```

Порядок важливий при spread об'єктів: властивості, вказані пізніше, перезаписують однойменні попередні. `{...a, ...b} ≠ {...b, ...a}`, якщо ключі повторюються.

Array.from() — перетворення псевдомасиву на масив

Псевдомасив (наприклад, `arguments` у функції) схожий на масив — має `length` і доступ за індексом, — але не має методів масиву (`push`, `slice`, `join` тощо). `Array.from()` створює з нього повноцінний масив.

```
function foo() {
  console.log(arguments); // псевдомасив [1, 2, 3]
  const args = Array.from(arguments); // повноцінний масив [1, 2, 3]
  return args.join("-"); // "1-2-3" – join() недоступний на arguments напряму
}
foo(1, 2, 3);
```

Із появою `...rest` потреба в `Array.from(arguments)` зменшилась — `function foo(...args)` одразу дає повноцінний масив. Але `Array.from()` лишається корисним для будь-якої іншої array-like структури (наприклад, результату `querySelectorAll` у роботі з DOM).

11. Швидкий довідник

Метод / синтаксис	Категорія	Призначення
<code>.length</code>	масив	кількість елементів
<code>.push()</code>	масив	додати в кінець (змінює оригінал)
<code>.slice()</code>	масив	копія частини масиву
<code>.concat()</code>	масив	об'єднати масиви в новий
<code>.indexOf()</code> / <code>.includes()</code>	масив	пошук елемента / перевірка наявності
<code>.join()</code> / <code>.split()</code>	масив / рядок	масив → рядок / рядок → масив
<code>for...of</code>	масив	перебір елементів (без індексу)
<code>for...in</code>	об'єкт	перебір ключів об'єкта
<code>Object.keys()</code> / <code>Object.values()</code>	об'єкт	масив ключів / масив значень
<code>this</code>	метод об'єкта	посилання на об'єкт, що викликав метод
<code>...rest</code>	функція (параметри)	збирає аргументи в масив
<code>...spread</code>	виклик / масив / об'єкт	розкладає масив/об'єкт на елементи або об'єднує
<code>Array.from()</code>	псевдомасив	перетворює array-like структуру (напр. <code>arguments</code>) на повноцінний масив
<code>===</code> на масивах/об'єктах	порівняння	порівнює посилання, не вміст — завжди <code>false</code> для різних екземплярів